eedi-mining-misconceptions-in-mathematics

2nd place solution

Rank	2
Team	CQYR
Author	RihanPiggy
Topic ID	551651
Version	Original

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551651
Retrieved with Kaggle CLI on 2026-10-21.

Congratulations to all the winners! Thanks to the organizers for hosting such an interesting competition. We really enjoyed our journey and got a lot of inspiration from all the competitors! I am lucky enough to be a member of such a fruitful team and would like to thank all of my team members. Here we share our solution.

Preprocess

We found a useful subject_metadata.csv of which the SubjectId and SubjectName are Identical to this competition from the past eedi competition hosted on NeurIPS 2022. The subject_metadata.csv contains the parent subject, so we made a vector db with this metadata to add the parent subject information to both train.csv and test.csv.

Synthetic data

synthetic questions

We generated synthetic data 3 times, let's call them generation1, generation2, generation3.
The base approach is:

- provide LLM a misconception with few examples and let it generate questions
- use qwen-math to solve the question to get the correct answer
- use qwen-math to solve the question under the constraint of misconception to get the wrong answer
- use gpt-4o-mini to score the quality of the question and choose those with score larger than 2 (max is 5)

The difference between each generation is as follows.
Generation1:

- few shot examples: randomly sampled from train.csv

Generation2:

- few shot examples: sample the question with the same misconception from train.csv and Generation1

Generation3:

- The prompt of generation 3 is based on the prompt of this tech blog
- few shot examples: randomly sample 2 questions from train.csv, Generation1 and Generation2

misconception augmentation

The misconception only contains a short sentence. In order to make the embedding of the misconception more meaningful, we used LLM to generate explanation for each misconception. Since we don't need to run inference for misconception in submission, the approach costs nothing in submission.
The prompt is as follows. The explanation of llama3.1-70b-Instruct and qwen2.5-72b-Instruct out-perform gpt-4o-mini in training retriever.

```
system_prompt_template =

user_prompt_template =
```

Chain of thought

We used qwen2.5-32B-Instruct-AWQ to generate chain-of-thought as additional input for the following retrieve and rerank. The prompt is as follows:

```
system_prompt_template =

user_prompt_template =
```

Retrieve

Training the Retrieve Models

We trained retriever with 2 different pipeline.
Pipeline1:

- backbone: Linq-AI-Research/Linq-Embed-Mistral
- loss: Arcface
- use Chain of thought as additional input in training and inference

Pipeline2:

- backbone: Qwen/Qwen2.5-14B, Qwen/Qwen2.5-32B, Qwen/QwQ-32B-Preview
- loss: MultipleNegativesRankingLoss
- w/o Chain of thought (due to inference time)

Single model performance is as follows.

retriever	synthetic data	Private LB	Public LB	Inference time
Linq-AI-Research/Linq-Embed-Mistral	Generation123	0.461	0.484	50 min
Qwen/Qwen2.5-14B	Generation12	0.479	0.507	45 min
Qwen/Qwen2.5-14B	Generation123	0.485	0.492	45 min
Qwen/Qwen2.5-32B	Generation123	0.495	0.536	140 min
Qwen/QwQ-32B-Preview	Generation123	0.500	0.531	140 min

Our best submission used an ensemble of Mistral and 2 x qwen2.5-14B to give enough time to 72b rerank, the private LB and public LB is 0.513, 0.530.

Key Factors for Retriever Improvements

1. **Using Large Models**
   As is shown in the table above, the larger the better, GPU and Credit Card is all you need to get Power!!! Never open the billing page during the competition.
   - qwen2.5-14B with Generation12: H100 about 2days
   - qwen2.5-14B with Generation123: H100 about 3days
   - qwen2.5-32B with Generation123(sampled): H100 about 5days
   - QWQ with Generation123(sampled): H100 about 5days
2. **Synthetic question**
   I believe most of the participants used synthetic questions, the more high quality questions, the better performance. For our team, using gpt-4o-mini to filter high quality questions is the key.
3. **Misconception augmentation**
   Using misconception augmentation significantly boosted retriever performance by about 2-4%.
4. **Chain of Thought**
   CoT is also useful. But for 14B and 32B models, adding CoT to the prompt will double the inference time.
5. **Pooling Selection**
   We found that last token pooling achieved better performance than mean pooling in the Qwen model.

Rerank

We used a listwise reranker to refine the ranking of retrieved candidates. Our reranking process employed a sliding window approach: first, we used a lightweight LLM to reorder candidates ranked between 8th and 17th. Then, we leveraged larger models to finalize the rankings for the top 10 candidates.

The LLMs for reranking were fine-tuned on a combination of synthetic and training data.

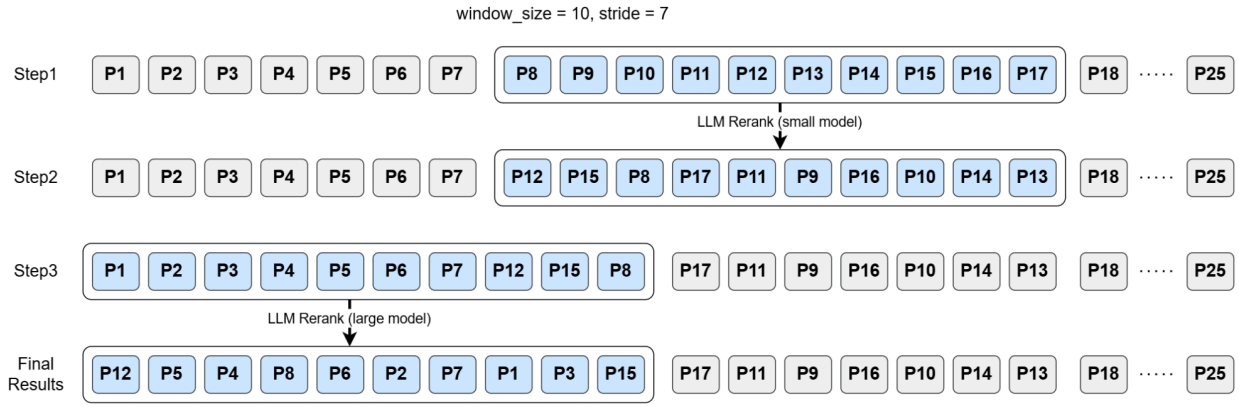
- **Window 1 (8th ~ 17th)**
  - Qwen2.5-14B-Instruct
- **Window 2 (1st ~ 10th)**
  - Qwen2.5-72B-Instruct
  - Llama-3.3-70B-Instruct

Key Factors for Reranking Improvements

1. **Using Large Models**
   We found that larger models (e.g., 72B parameters) consistently delivered stronger validation scores compared to smaller ones like 14B or 32B models. However, these larger models initially performed worse on the Public LB, leading to some concerns. Despite this, we trusted the validation scores and included the 72B model in our final submissions (special thanks to the three-submission rule!). Ultimately, the 72B model produced outstanding Private Leaderboard scores, helping us secure a prize.

2. **Chain of Thought**
   The above CoT prompts greatly improved reranking performance.

3. **Sliding Window**
   Instead of increasing the number of candidates for reranking, applying the sliding window strategy multiple times to refine the top-10 rankings proved to be more effective.



4. **Test-Time Augmentation**
   During inference, we used TTA with some models by generating prompts in reverse order and averaging their scores with those from standard prompts. This technique provided a slight boost in accuracy.

Training the Rerank Models

We developed the QLoRA training code for our LLM rerankers based on the first-place solution from atmacup17. Special thanks to @kcotton21 for sharing such excellent solutions.

Here are some tips that proved effective during the reranking model training:

1. **Randomizing Listwise Choices**
   Instead of always using the top-10 candidates for prompts, we created prompts with a variety of top-N combinations such as top-3, top-5, top-15, and top-25.

2. **Synthetic Data**
   The synthetic data used for training the retriever was also helpful for training the reranker. In total, we trained on 8,000 records of training data (2 epochs) and 14,000 synthetic records, resulting in a combined dataset of 22,000 records.

3. **Negative Sample Mining**
   To mine negative samples, we used a hybrid retriever combining the fine-tuned `dunzhang/stella_en_400M_v5` model and TF-IDF. Each of the 22,000 positive samples had corresponding negative samples mined using this setup.

Training Time

Qwen2.5-14B: ~2 hours on H100
Qwen2.5-72B: ~8 hours on H100

Quantization

We used the intel/auto-round library for quantizing the LLM rerankers. Compared to AutoGPTQ and AutoAWQ, this library was easier to use and caused minimal accuracy loss (typically less than 2%). Additionally, it could produce models compatible with vLLM.

qwen2.5-72b-Instruct have some issues to run on multi GPU due to its intermediate_size(29568). Following the workaround provided by the document of gptq, we padded the weights to 29696 and then performed quantization.

For calibration, we used the training dataset. Below are the quantization parameters:

```
bits, group_size, sym = 4, 128,
autoround = AutoRound(
    model, tokenizer, =bits, =group_size, =sym, =calib_prompts, =256,
    =512,
    =500,
)
```

Inference

We use vLLM for inference.

By setting the `enabling_prefix_cache` to `True`, we were able to save approximately 10% of the inference time.

jagatkiran shared his insights on performing inference with a 72B LLM model. In this competition, larger models tend to perform better, which has been very helpful for us.

We implemented the reranker using `logits_processors` and `logprobs` by assigning a weight of +100 to specific tokens. This approach helped us establish the framework for the ranker. We also tried using a classification head directly, but the results were not satisfactory, and it was not easy to perform inference with vLLM. We believe this method could become a paradigm in future competitions.

Ablation

Baseline (retriever)	Qwen14B	Qwen72B	Llama70B	8-17th Qwen14B	Private LB	Public LB
✅					0.513	0.530
✅	✅				0.568	0.583
✅		✅			0.593	0.582
✅	✅	✅	✅		0.596	0.609
✅	✅	✅	✅	✅	**0.604**	0.622

On the final day, we tried fine-tuning Nexusflow/Athene-V2-Chat instead of Llama70B. Unfortunately, the submission with this model got timeout due to gpu and timeout issues, but it showed highly impressive performance on the leaderboard: 0.609.

train code: https://github.com/wangqihanginthesky/Eedi_kaggle\
inference code: https://www.kaggle.com/code/honglihang/2nd-place-inference-code

Reference

LLMにおける合成データセットによる数学推論タスクの精度向上の検討
NeurIPS 2022 CausalML Challenge: Causal Insights for Learning Paths in Education
A Setwise Approach for Effective and Highly Efficient Zero-shot Ranking with Large Language Models
Qwen2.5-72B & Llama 3.3-70B on 2xT4(questionable performance)
Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs
1st place solution from atmacup17
Winning Amazon KDD Cup'24
Qwen GPTQ Troubleshooting

Supplemental Q&A

> **Yi-Chia Chen・2024-12-14T17:26:21.777000**
>
> > "We found a useful subject_metadata.csv of which the SubjectId and SubjectName are Identical to this competition from the past eedi competition hosted on NeurIPS 2022. The subject_metadata.csv contains the parent subject, so we made a vector db with this metadata to add the parent subject information to both train.csv and test.csv."
>
> That's cool! Have you analyzed how much improvement the parent subject information brings to the model's performance?

> **kuto・2024-12-15T01:04:23.557000**
>
> Congratulations and thank you for sharing your solution! It was very interesting to read.
>
> I have two questions.
>
> 1. Are the prompt and output formats different between training and inference for the reranker? I understood it as predicting Yes/No for a single misconception during training, and rearranging multiple misconceptions during inference. Is that correct?
> 2. I didn’t quite understand the part where the special token is assigned a weight of 100. Could you explain that in a bit more detail?
>
> > We implemented the reranker using logits_processors and logprobs by assigning a weight of +100 to specific tokens.

> **ktr・2024-12-15T02:26:57.653000**
>
> Congratulations on your prize gold! I am glad to hear that my past solutions were helpful.

I can see from your post that you have made good use of 72b.
Did you train the 72b with QLoRA, merge the Adapter, and then quantize it? Empirically, I think that merge Adapter trained with QLoRA has a significant performance degradation.

Qihang Wang · 2024-12-15T03:36:58.233000

Thank you for your question.

The training and inference processes are consistent, both involving options like A, B, C, D …

Regarding the weight of the special token: for these options (A, B, C, D,…), we apply a logit_bias of +100 to ensure that the model prioritizes these letters as output. (set `scores[self.allowed_ids] += 100` in logitprocessor's **call**)

RihanPiggy · 2024-12-15T04:05:18.547000

Thanks very much for your solution in atmacup and congratulations!
Yes, we trained 72b with QLoRa, merge the Adapter and then perform quantization. In our offline experiment, the performance did not show much degradation(0.7011 -> 0.6988).
@yokuyama developed the training of rerank and I heard that he optimized the hyper-parameter during quantize. auto-round is very powerful and easy to use, better than auto-awq in our experiments.

RihanPiggy · 2024-12-15T04:10:21.173000

Not sure enough because I collected external data at the start of the competition. Personally I think the gain is marginal because our 14B and 32B retriever did not use this information. However, the parent subject is used as input of Chain-of-thought, Rerank and synthetic data generation, which may give the model some additional context.